

INFORME DE PRUEBAS DE SOFTWARE

Pruebas unitarias, de dominio y de integración

Parkings Together

Marketplace P2P de estacionamientos

Documento	Informe de pruebas de software
Proyecto	Parkings Together — Marketplace P2P de estacionamientos
Versión	2.0
Fecha	12 de junio de 2026
Asignatura	DSY1106
Docente	
Integrantes	
Resultado global	77 / 77 pruebas aprobadas (100 %)

Tabla de contenido

1. Introducción y objetivo	2
2. Alcance y entorno de pruebas	2
3. Estrategia y tipos de prueba	2
4. Resumen de resultados	3
5. Detalle por suite de pruebas	3
5.1. pricing.test.js — Cálculo de precios (24 casos)	3
5.2. payments.test.js — Proveedores de pago, patrón Strategy (30 casos)	4
5.3. geocoding.test.js — Regiones de Chile (8 casos)	4
5.4. rateLimit.test.js — Limitación de peticiones (5 casos)	4
5.5. api.test.js — Contratos del BFF (node:test, 6 casos)	4
5.6. reserveService.test.js — Saga de reservas (2 casos)	4
5.7. api.timeout.test.js — Resiliencia del BFF (2 casos)	5
6. Cobertura de código	5
7. Evidencia de ejecución	6
8. Cómo reproducir las pruebas	6
9. Alcance, limitaciones y mejoras futuras	7
10. Conclusiones	7

Nota: en Microsoft Word, haga clic derecho sobre el índice y seleccione «Actualizar campos» para numerar las páginas.

1. Introducción y objetivo

El presente informe documenta las pruebas automatizadas implementadas para el proyecto Parkings Together, una plataforma tipo marketplace de intermediación (P2P) para el arriendo de estacionamientos entre particulares. El objetivo es verificar, mediante pruebas reproducibles, que la lógica de negocio crítica y los contratos de las interfaces (API) del sistema se comportan según lo esperado y se mantienen estables ante cambios.

Las pruebas se ejecutan de forma aislada, sin depender de servicios externos: tanto la base de datos como la red se sustituyen por dobles de prueba (mocks). De este modo, las pruebas son rápidas, deterministas y reproducibles en cualquier entorno, lo que las hace aptas como evidencia objetiva de la calidad del software.

2. Alcance y entorno de pruebas

El alcance cubre la lógica de negocio pura (cálculo de precios, proveedores de pago, geocodificación de regiones de Chile, limitación de peticiones), la lógica de dominio del microservicio de reservas (patrón Saga con compensación y control de capacidad bajo CQRS) y el contrato del orquestador de servicios (BFF). La siguiente tabla resume el entorno técnico.

Aspecto	Detalle
Lenguaje / runtime	JavaScript · Node.js (v24; requiere ≥ 22)
Pruebas unitarias y de dominio	Jest (babel-jest para módulos ESM)
Pruebas de integración (BFF)	node:test — runner de pruebas nativo de Node.js
Cobertura de código	Istanbul (reporte de texto y HTML lcov-report)
Aislamiento	Mock de fetch global y mock de repositorio (jest.mock): sin base de datos ni red reales
Ubicación de las pruebas	apps/web/tests/ y apps/ms-reservas/tests/

3. Estrategia y tipos de prueba

Se aplicó una estrategia que combina pruebas unitarias de funciones puras de negocio, pruebas de dominio y pruebas de integración del contrato de las interfaces. Cada tipo valida una preocupación distinta del sistema:

Tipo	Qué valida	Suites
Unitarias (negocio)	Funciones puras: precios, pagos, regiones, rate limit	pricing · payments · geocoding · ratelimit
Dominio / Saga	Reserva atómica con compensación (rollback) y capacidad (CQRS)	reserveService.test.js
Integración de API (BFF)	Contrato de las rutas del orquestador (estructura y errores)	api.test.js
Resiliencia	Timeout estricto y respuesta de respaldo (fallback) del BFF	api.timeout.test.js

4. Resumen de resultados

Se ejecutaron 77 pruebas automatizadas distribuidas en 7 suites. La totalidad resultó aprobada, en un tiempo de ejecución de pocos segundos y sin dependencias externas.

Suite	Framework	Casos	Resultado
pricing.test.js	Jest	24	24 / 24 aprobadas
payments.test.js	Jest	30	30 / 30 aprobadas
geocoding.test.js	Jest	8	8 / 8 aprobadas
rateLimit.test.js	Jest	5	5 / 5 aprobadas
reserveService.test.js	Jest	2	2 / 2 aprobadas
api.timeout.test.js	Jest	2	2 / 2 aprobadas
api.test.js (BFF)	node:test	6	6 / 6 aprobadas
TOTAL	—	77	77 / 77 (100 %)

Las pruebas Jest cubren 71 casos en 6 suites; la suite de integración del BFF (api.test.js) aporta 6 casos adicionales mediante el runner nativo node:test.

5. Detalle por suite de pruebas

5.1. pricing.test.js — Cálculo de precios (24 casos)

Módulo: apps/web/src/lib/pricing.js (calcTotal, calcBreakdown). Valida el cálculo de tarifa por duración: duración 0 → \$0; estacionamiento gratuito → \$0; tarifas por hora,

por minuto y por día (incluyendo redondeos y fracciones); combinaciones día+hora+minuto; y que el desglose (calcBreakdown) sume exactamente lo mismo que calcTotal.

5.2. payments.test.js — Proveedores de pago, patrón Strategy (30 casos)

Módulo: apps/web/src/lib/payments.js. Valida isValidProvider (acepta mock/efectivo/webpay y rechaza otros), genTransactionId (formato, prefijo y unicidad), isWebpayConfigured, y que createCharge devuelva un contrato uniforme { status, transactionId, raw } para los distintos proveedores.

5.3. geocoding.test.js — Regiones de Chile (8 casos)

Módulo: apps/web/src/lib/comunas-chile.js (REGIONES, detectarRegion). Valida que existan las 16 regiones con sus campos y límites coherentes, que coordenadas reales caigan en la región correcta (Santiago → Región Metropolitana, Valparaíso → V Región) y que puntos fuera de Chile devuelvan null.

5.4. rateLimit.test.js — Limitación de peticiones (5 casos)

Módulo: apps/web/src/lib/rateLimit.js. Valida el limitador de tasa: permite hasta el máximo de accesos y bloquea el siguiente, mantiene contadores independientes por clave y decrementa el remanente en cada acceso; además, clientIp extrae correctamente la IP del cliente desde la cabecera x-forwarded-for.

5.5. api.test.js — Contratos del BFF (node:test, 6 casos)

Prueba la estructura de respuesta de las rutas de mismo origen /api/mapas/search y /api/reservas/reserve con un mock de fetch, sin base de datos real. Casos cubiertos:

#	Caso	Resultado
1	Mapas: actualiza la ocupación correctamente	Aprobado
2	Mapas: rechaza la actualización al exceder la capacidad (PATCH)	Aprobado
3	Reservas: verifica disponibilidad cuando hay plaza libre	Aprobado
4	Reservas: verifica disponibilidad cuando la plaza está llena	Aprobado
5	Reservas: crea la reserva con éxito	Aprobado
6	Reservas: maneja el error cuando la Saga falla	Aprobado

5.6. reserveService.test.js — Saga de reservas (2 casos)

Módulo: `apps/ms-reservas/.../services/reserva.service.js`. Demuestra la robustez transaccional mediante el patrón Saga con compensación:

- **Control de capacidad (CQRS):** rechaza la reserva si el estacionamiento está lleno y, en ese caso, no llega a crear la reserva.
- **Compensación (rollback):** si falla el aumento de ocupación (error de concurrencia), la Saga revierte la reserva creada, garantizando la integridad de los datos.

5.7. `api.timeout.test.js` — Resiliencia del BFF (2 casos)

Verifica que el cliente del BFF devuelva los datos cuando la respuesta llega a tiempo, y que aborte la petición mediante `AbortController` entregando una respuesta de respaldo segura (`{ success: false, error: true, fallback: true, data: [] }`) cuando se supera el límite estricto de 8 segundos.

6. Cobertura de código

La siguiente tabla presenta la cobertura global del código probado, generada con Istanbul (`npx jest --coverage`). La lógica de negocio pura alcanza una cobertura muy alta, y las ramas (branches) del servicio de reservas —donde reside la Saga— están cubiertas al 100 %.

Archivo	% Stmts	% Branch	% Funcs	% Lines
All files	73.15	70.43	41.46	72.85
<code>pricing.js</code>	100	97.5	100	100
<code>payments.js</code>	92.30	80	100	92.30
<code>comunas-chile.js</code>	100	100	100	100
<code>rateLimit.js</code>	100	81.81	100	100
<code>reserva.service.js</code>	78.57	100	50	78.57
<code>api.js</code>	39.58	25	12.50	38.63

La cobertura global es del 73.15 % de sentencias. La lógica de negocio (`pricing`, `payments`, `comunas-chile`, `rateLimit`) está prácticamente cubierta en su totalidad. El orquestador `api.js` presenta una cobertura menor (39.58 %) porque incorpora numerosas rutas y ramas de error que se ejercitan parcialmente; ampliarla es la principal oportunidad de mejora (ver sección 9).

7. Evidencia de ejecución

A continuación se transcribe la salida real de la ejecución, de modo que el informe es autocontenido y verificable. Las capturas pueden regenerarse ejecutando los comandos de la sección 8.

Evidencia 1 — Suite Jest con cobertura (71 casos):

```
PS (raiz)> npx jest --coverage
PASS  apps/web/tests/pricing.test.js
PASS  apps/web/tests/payments.test.js
PASS  apps/web/tests/geocoding.test.js
PASS  apps/web/tests/ratelimit.test.js
PASS  apps/web/tests/api.timeout.test.js
PASS  apps/ms-reservas/tests/reserveService.test.js
-----|-----|-----|-----|
File      | % Stmts | % Branch | % Funcs | % Lines |
-----|-----|-----|-----|
All files | 73.15   | 70.43    | 41.46    | 72.85    |
-----|-----|-----|-----|
Test Suites: 6 passed, 6 total
Tests:       71 passed, 71 total
```

Evidencia 1: 6 suites y 71 pruebas Jest aprobadas, con reporte de cobertura global 73.15 %.

Evidencia 2 — Integración del BFF (node:test, 6 casos):

```
PS apps/web> node --test tests/api.test.js
ok 1 - BFF - Mapas: actualizar ocupacion correctamente
ok 2 - BFF - Mapas: falla al exceder capacidad en PATCH
ok 3 - BFF - Reservas: verificar disponibilidad (plaza libre)
ok 4 - BFF - Reservas: verificar disponibilidad (plaza llena)
ok 5 - BFF - Reservas: crear reserva con exito
ok 6 - BFF - Reservas: maneja error si falla la creacion
i tests 6 | i pass 6 | i fail 0
```

Evidencia 2: 6/6 pruebas del BFF aprobadas (node:test).

8. Cómo reproducir las pruebas

Desde la raíz del monorepo, tras instalar las dependencias, las pruebas se reproducen con los siguientes comandos:

```
# Instalar dependencias (una sola vez)
npm install

# 1) Suite de negocio, dominio y resiliencia (Jest) + cobertura -> 71 pruebas
npx jest --coverage # genera coverage/lcov-report/index.html

# 2) Integración del BFF (node:test) -> 6 pruebas
cd apps/web
npm test # ejecuta: node --test tests/api.test.js
```

El reporte HTML de cobertura queda disponible en `coverage/lcov-report/index.html` para su revisión en el navegador.

9. Alcance, limitaciones y mejoras futuras

En aras de la transparencia, se documentan los límites actuales y las líneas de trabajo recomendadas:

- La cobertura del orquestador api.js (39.58 % de sentencias, 12.5 % de funciones) es baja porque concentra numerosas rutas (favoritos, pagos, premium, reseñas, soporte) que aún no tienen pruebas dedicadas. Es la principal oportunidad de mejora.
- Se recomienda añadir pruebas de contrato para el resto de las rutas del BFF, de modo de elevar la cobertura de funciones por encima del 60 % en todos los componentes.
- Se recomienda incorporar pruebas de extremo a extremo (end-to-end) del flujo de reserva completo, una vez disponible un entorno de integración con base de datos de prueba.

10. Conclusiones

Las 77 pruebas automatizadas se ejecutan de forma satisfactoria (100 % aprobadas), de manera determinista y sin dependencias externas, validando los componentes centrales del sistema.

La lógica crítica de negocio está muy cubierta: precios, pagos, geocodificación y limitación de peticiones alcanzan cobertura cercana al 100 %. La robustez transaccional queda demostrada: la Saga del microservicio de reservas valida la capacidad (CQRS) y revierte ante fallos (compensación, con ramas al 100 %), y el orquestador (BFF) gestiona de forma controlada los errores y la latencia mediante timeout y respuesta de respaldo.

En conjunto, la batería de pruebas constituye evidencia objetiva de que los componentes centrales de Parkings Together se comportan según lo esperado y quedan protegidos ante regresiones, dejando una base clara para ampliar la cobertura a las áreas señaladas en la sección 9.